

GitHub Actions CI/CDガイド

EC2の環境設定

Nginxのインストールと起動状態とバージョン確認

1. インストール

```
sudo apt update
sudo apt install -y nginx
```

2. 起動状態とバージョンを確認

```
sudo systemctl status nginx
nginx -v
```

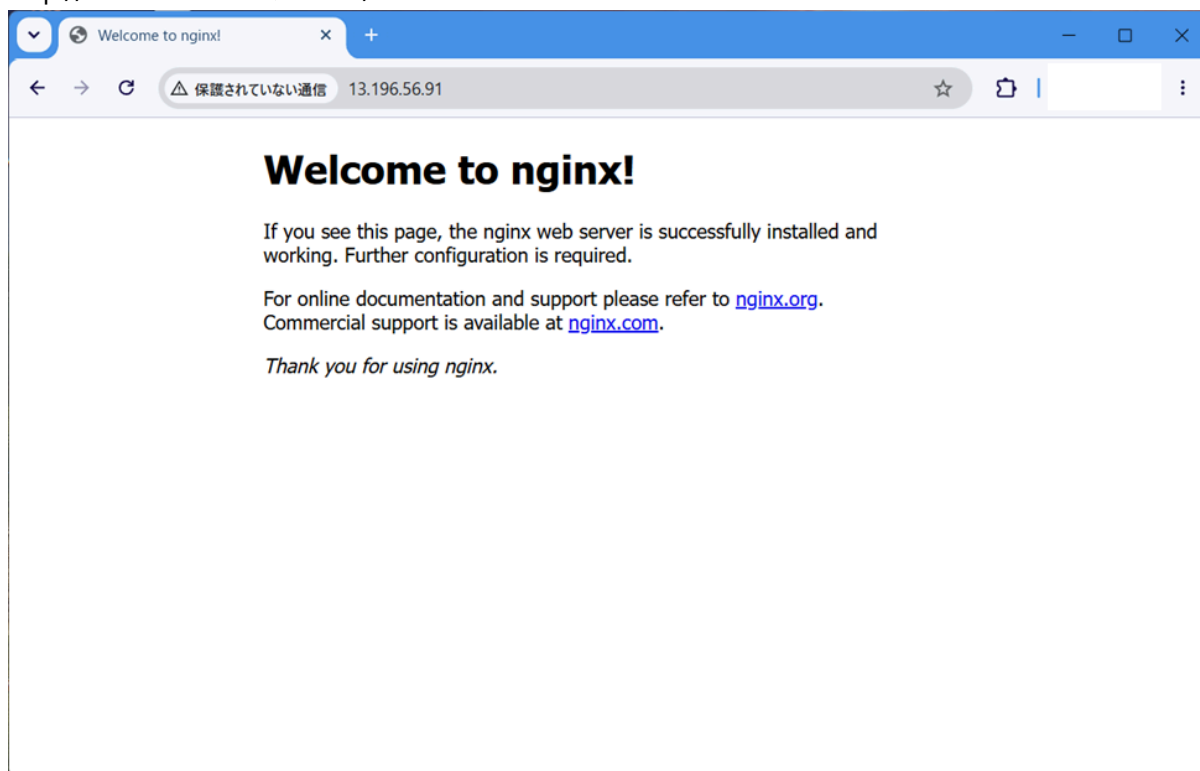
```
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset:
   enabled)
   Active: active (running) since Tue 2026-07-21 23:36:48 UTC; 59s ago
   Invocation: ee09e37a499e4b1fb40e84fcb12cb20d
     Docs: man:nginx(8)
   Process: 30063 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process
   on; (code=exited, status=0/SUCCESS)
   Process: 30065 ExecStart=/usr/sbin/nginx -g daemon on; master_process on;
   (code=exited, status=0/SUCCESS)
  Main PID: 30094 (nginx)
    Tasks: 3 (limit: 3736)
   Memory: 3M (peak: 7.2M)
      CPU: 45ms
   CGroup: /system.slice/nginx.service
           └─30094 "nginx: master process /usr/sbin/nginx -g daemon on;
master_process on;"
             └─30097 "nginx: worker process"
               └─30098 "nginx: worker process"
```

```
Jul 21 23:36:48 ip-10-0-0-34 systemd[1]: Starting nginx.service - A high
performance web server and a reverse proxy server...
Jul 21 23:36:48 ip-10-0-0-34 systemd[1]: Started nginx.service - A high
performance web server and a reverse proxy server.
```

```
nginx version: nginx/1.28.3 (Ubuntu)
```

3. デフォルトページへのアクセス確認

- `http://<EC2のパブリックIP>/`



バックエンド用設定ファイルの作成

1. ファイルの作成

- tee で一気に書き込む方法(リバースプロキシ設定 (管理+顧客))
- ファイル名:fullness-ec-exercise.conf

```
# 新名称で作成(統合設定)
sudo tee /etc/nginx/sites-available/fullness-ec-exercise.conf > /dev/null <<'EOF'
server {
    listen 80;
    server_name _;

    # 管理側(バックエンド) : /admin 配下
    location /admin {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # 顧客側(フロントエンド) : それ以外すべて
    location / {
        proxy_pass http://127.0.0.1:8082;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
}  
}  
EOF
```

2. サイトを有効化し、デフォルトサイトを無効化する

```
sudo ln -sf /etc/nginx/sites-available/fullness-ec-exercise.conf /etc/nginx/sites-enabled/  
sudo rm -f /etc/nginx/sites-enabled/default  
sudo nginx -t  
sudo systemctl reload nginx
```

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok  
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

systemdサービスの作成

1. 環境変数ファイルの作成

- /etc/fullness/backend.envの作成
- RDSのエンドポイントは実値に置き換えてください。

```
sudo useradd --system --no-create-home --shell /usr/sbin/nologin fullness  
sudo mkdir -p /etc/fullness  
sudo mkdir -p /opt/fullness  
  
sudo tee /etc/fullness/backend.env > /dev/null << 'EOF'  
DB_URL=jdbc:postgresql://<RDSエンドポイント>:5432/fullness-ec  
DB_USERNAME=postgres  
DB_PASSWORD=training  
AWS_REGION=ap-northeast-1  
AWS_S3_BUCKET=fullness-ec-product-images-furukawa  
EOF  
  
sudo chmod 600 /etc/fullness/backend.env  
sudo chown fullness:fullness /etc/fullness/backend.env
```

2. サービス定義の作成

- /etc/systemd/system/fullness-backend.service

```
sudo tee /etc/systemd/system/fullness-backend.service > /dev/null << 'EOF'  
[Unit]  
Description=Fullness Stationery Backend (Spring Boot)  
After=network.target
```

```
[Service]
User=fullness
Group=fullness
EnvironmentFile=/etc/fullness/backend.env
ExecStart=/usr/bin/java -jar /opt/fullness/backend.jar
SuccessExitStatus=143
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF
```

3. 起動と有効化

```
sudo systemctl daemon-reload
sudo systemctl enable fullness-backend
```

Actions secrets and variables

キー	値
EC2_HOST	Elastic IP
EC2_USER	ubuntu
EC2_SSH_KEY	キー.pem

CI/CDワークフロー

- backend-cicd.yml

```
name: Backend CICD

on:
  push:
    branches: [ main ]
    paths:
      - 'backend-answer/**'
      - '.github/workflows/backend-cicd.yml'
  pull_request:
    paths:
      - 'backend-answer/**'

jobs:
  build:
    runs-on: ubuntu-latest
    services:
      postgres:
```

```
image: postgres:17
env:
  POSTGRES_DB: fullness-ecx
  POSTGRES_USER: postgres
  POSTGRES_PASSWORD: training
ports:
  - 5432:5432
options: >-
  --health-cmd "pg_isready -U postgres -d fullness-ecx"
  --health-interval 10s
  --health-timeout 5s
  --health-retries 5
steps:
  - name: チェックアウト
    uses: actions/checkout@v4
  - name: JDK 17 のセットアップ
    uses: actions/setup-java@v4
    with:
      distribution: temurin
      java-version: '17'
  - name: データベース初期化(スキーマ+シード)
    env:
      PGPASSWORD: training
    run: |
      psql -h localhost -U postgres -d fullness-ecx -f backend-
answer/src/main/resources/sql/create_tables.sql
      psql -h localhost -U postgres -d fullness-ecx -f backend-
answer/src/main/resources/sql/insert_data.sql
  - name: gradlew に実行権限を付与
    working-directory: backend-answer
    run: chmod +x ./gradlew
  - name: ビルド&テスト
    working-directory: backend-answer
    env:
      DB_URL: jdbc:postgresql://localhost:5432/fullness-ecx
      DB_USERNAME: postgres
      DB_PASSWORD: training
    run: ./gradlew build

deploy:
  needs: build
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - name: チェックアウト
      uses: actions/checkout@v4
    - name: JDK 17 のセットアップ
      uses: actions/setup-java@v4
      with:
        distribution: temurin
        java-version: '17'
    - name: 実行用JARをビルド
      working-directory: backend-answer
      run: |
```

```
    chmod +x ./gradlew
    ./gradlew clean bootJar
- name: JARをEC2へ転送
  uses: appleboy/scp-action@v0.1.7
  with:
    host: ${ secrets.EC2_HOST }
    username: ${ secrets.EC2_USER }
    key: ${ secrets.EC2_SSH_KEY }
    source: "backend-answer/build/libs/*-SNAPSHOT.jar"
    target: "/tmp/"
    strip_components: 3
- name: 配置してサービス再起動
  uses: appleboy/ssh-action@v1.0.3
  with:
    host: ${ secrets.EC2_HOST }
    username: ${ secrets.EC2_USER }
    key: ${ secrets.EC2_SSH_KEY }
    script: |
      sudo mv /tmp/*-SNAPSHOT.jar /opt/fullness/backend.jar
      sudo chown fullness:fullness /opt/fullness/backend.jar
      sudo systemctl restart fullness-backend
      sleep 5
      sudo systemctl is-active fullness-backend
```

EC2 への S3 アクセス権限設定（IAM ロール）手順

目的・背景

バックエンドは、商品画像の表示に **S3 の署名付き URL (presigned URL)** を生成する。この署名には AWS 認証情報が必要だが EC2 上で **fullness** サービスユーザーとして稼働しているため **~/.aws/credentials** は存在しない。

AWSSDKの **DefaultCredentialsProvider** は次の順で認証情報を探す。

1. 環境変数 (**AWS_ACCESS_KEY_ID** 等)
2. システムプロパティ
3. **~/.aws/credentials** (プロファイル)
4. コンテナ認証情報
5. **EC2 インスタンスプロファイル (IAM ロール、IMDS 経由)**

稼働環境では1~4 が無いため、**5 の IAM ロールを EC2 に割り当てる**ことで認証情報を供給する。ロール未割り当てだと、商品検索時に次の例外が発生する。

```
software.amazon.awssdk.core.exception.SdkClientException:
  Unable to load credentials ... InstanceProfileCredentialsProvider(): Failed to
  load credentials from IMDS.
  at S3ImageStorage.generatePresignedUrl(...)
```

全体の流れ

1. IAM ポリシーを作成（S3 の必要な権限を定義）
2. IAM ロールを作成（信頼されたエンティティ = EC2）し、ポリシーをアタッチ
3. EC2 インスタンスにロールを割り当て
4. サービスを再起動して認証情報を反映
5. 動作確認

1. IAM ポリシーの作成

IAM → **ポリシー** → **ポリシーを作成** → **JSON** タブに以下を貼り付ける。

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::fullness-ec-product-images-furukawa/*"
    }
  ]
}
```

- **s3:GetObject** ... 署名付き URL で画像を**取得**するために必要（商品一覧・詳細表示）
- **s3:PutObject** ... 商品登録・修正で画像を**アップロード**するために必要
- **Resource** ... 対象バケットのオブジェクト（末尾 **/*** はバケット内の全オブジェクトを指す） 「次へ」
→ ポリシー名（例：**FullnessS3Access**）を入力 → **ポリシーの作成**。

バケット名（**fullness-ec-product-images-furukawa**）が実際のバケットと一致していることを確認する。

2. IAM ロールの作成

IAM → **ロール** → **ロールを作成**。

1. **信頼されたエンティティタイプ**：**AWS のサービス**
2. **ユースケース**：**EC2**
3. 「次へ」 → 許可ポリシーで **FullnessS3Access** を検索してチェック
4. 「次へ」 → ロール名（例：**FullnessEC2Role**）を入力 → **ロールを作成** これで「EC2 が引き受け可能で、S3 読み書き権限を持つロール」ができる。

3. EC2 インスタンスへロールを割り当て

EC2 → インスタンス → 対象インスタンスを選択 → アクション → セキュリティ → IAM ロールを変更
→ **FullnessEC2Role** を選択 → 更新。割り当ては即時反映され、以降インスタンスは IMDS 経由でロールの一時認証情報を取得できるようになる。

4. サービスの再起動

アプリケーションを再起動して認証情報を取り直す。

```
sudo systemctl restart fullness-backend
```

5. 動作確認

ブラウザで「商品情報メンテナンス（商品検索）」を開く。

- エラー画面ではなく商品一覧が表示されれば成功（＝署名付き URL 生成が通り、認証情報が取得できている）
- さらに「商品登録」で画像付きの商品を1件登録すると、アップロード（PutObject）と表示（GetObject）の両方を本番で確認できる うまくいかない場合はログを確認する。

```
sudo journalctl -u fullness-backend -n 100 --no-pager
```

- Failed to load credentials from IMDS ... ロール未割り当て or 再起動漏れ
- AccessDenied ... ロールは効いているが権限（アクション／バケット）が不足

補足

- シードデータの商品（初期5件）の image_url は /images/pen_black.png のような**仮のキー**であり、実オブジェクトが S3 に無いため、一覧では画像が壊れて表示される（ページ自体は正常）。実オブジェクトを置くか image_url を実在キーに更新すれば表示される。
- IAM ロール方式は、アクセスキーをサーバーに置かず済むため、キー漏洩リスクが無く安全。一時認証情報は自動でローテーションされる。
- ローカル開発では ~/.aws/credentials (aws configure で設定) が使われるため、この IAM ロールは本番（EC2）専用。